

```

Int64Index: 51 entries, 1 to 364
Data columns:
EDT                    51 non-null values
Max TemperatureF       51 non-null values
Mean TemperatureF      51 non-null values
Min TemperatureF       51 non-null values
Max Dew PointF         51 non-null values
Mean Dew PointF        51 non-null values
Min DewpointF          51 non-null values
Max Humidity           51 non-null values
Mean Humidity          51 non-null values
Min Humidity           51 non-null values
Max Sea Level PressureIn 51 non-null values
Mean Sea Level PressureIn 51 non-null values
Min Sea Level PressureIn 51 non-null values
Max VisibilityMiles     51 non-null values
Mean VisibilityMiles    51 non-null values
Min VisibilityMiles     51 non-null values
Max Wind SpeedMPH      51 non-null values
Mean Wind SpeedMPH     51 non-null values
Max Gust SpeedMPH      51 non-null values
PrecipitationIn        51 non-null values
CloudCover             51 non-null values
Events                 23 non-null values
WindDirDegrees         51 non-null values

```

```
dtypes: float64(4), int64(16), object(3)
```



Note: The output of the condition `data['Mean TemperatureF']>40` and `data['Mean TemperatureF']<50` return a NumPy array, and we must use the brackets to separate them before using the '&' operator, or else you will get an error message saying that the expression is ambiguous.

Now you can easily get meaningful data from your dataframe by simply filtering out the data that you aren't interested in. This provides you with a very powerful technique that you can use in conjunction with higher Pandas functions to understand your data.

Getting data out

You can easily write data out by using the `to_csv` function to write your data out as a csv file.

```
data.to_csv("weather-mod.csv")
```

Want to make a separate tab? No problem.

```
data.to_csv("data/weather-mod.tsv", sep="\t")
```



Note: Generally, the dataframe can be indexed by any Boolean NumPy array. In a sense, only values that are true will be retained. For example, if we use the variable `e`, (`e = data['Events'].apply(lambda d: pandas.isnull(d))`) which contains the list of all rows that have NaN values for `data['Events']`, as `data[e]`, we will get a dataframe which has rows that only have NaN values for `data['Events']`

You can also write your data out in other formats that can be found on the Pandas doc.

This article only covers the basics of what can be done with Pandas. It also supports a lot of higher level functions like plotting data to give a better feel of the data being dealt with. If you want to learn more about Pandas, check the online documentation. It is very readable, user-friendly and is a great place to get a better understanding of how Pandas works. Frameworks like Pandas let a Python application take advantage of such data analysis tools easily. There are also other languages which support data analysis and you may want to check them out. These include R, MATLAB, Julia and Octave. To wrap up, these languages and packages greatly increase your understanding of data. In a world where data is becoming increasingly important, it is critical that we deal with our data smartly. **END** 

By: Sahil Chelaramani

The author is an open source activist who loves Linux, Python and Android. He is a fourth year student at the Goa Engineering College (GEC). Beyond anything else, he loves to code.

India's #1 Website For Electronics Engineers Working Across The Globe

Established: 1998 | Registered Users: 300,000+



Monthly Statistics

Page Impressions:
750,000+

Unique Techies:
150,000



electronicsforu.com

If it's electronics, it's here

For any queries, please contact our team at efyeng@efyindia.com OR +91-11-26810001